

DJANGO · DRF · CHANNELS — MONOLITO MODULAR

Arquitectura del backend de Letdem

Cómo está construido el backend que sostiene el aparcamiento colaborativo, las alertas viales, el programa de puntos y el marketplace — y cómo lo desplegarías en un VPS. Un recorrido de sus capas, sus dominios, sus flujos de dinero y su operación.

1710 tests passed · 0 failed

14 apps de dominio

Python 3.11 · PostgreSQL/PostGIS

API /api/v1/

01 · Capas 02 · Dominios 03 · Dinero vs puntos 04 · Flujos

05 · Capas transversales 06 · Despliegue en VPS 07 · Credenciales / API keys

01 Vista de capas

Antes del diagrama, la idea de fondo: esto es un **monolito modular**. "Monolito" porque es un único proyecto Django que se despliega de una pieza; "modular" porque por dentro está partido en **apps de dominio** independientes (una por área de negocio) que se comunican por código, no por red. Es lo contrario a microservicios: más simple de operar, y suficiente para el tamaño actual del producto.

Lo particular de este backend es que **habla dos protocolos a la vez**. La mayoría de las peticiones son HTTP normal y corriente (pedir plazas, canjear puntos, pagar) y las sirve **gunicorn**, un servidor WSGI —síncrono, una petición entra, una respuesta sale—. Pero el mapa en tiempo real necesita **WebSockets** (una conexión que queda abierta y por la que el servidor empuja actualizaciones), y eso lo sirve **daphne**, un servidor ASGI asíncrono. Los dos corren en paralelo detrás de **nginx**, que hace de recepcionista: manda /api/ a gunicorn y /ws/ a daphne.

La tercera pata es el **trabajo en diferido**. Cosas que no deben bloquear la respuesta al usuario —enviar un email, comprimir una foto, caducar puntos por la noche— se encolan en **Celery** y se ejecutan aparte. Y atándolo todo, **Redis** hace un triple papel: cola de mensajes de Celery, transporte de los WebSockets (channel layer) y caché/sesiones.

Fig. 1 – Petición → nginx → runtime (WSGI / ASGI / Celery) → apps de dominio → datos + integraciones externas.

02 Mapa de dominios

Si la vista de capas responde "¿por dónde pasa una petición?", esta responde "¿de qué trata el negocio?". Cada app es un **contexto acotado**: un trozo del dominio con sus propios modelos y reglas. Entender las *relaciones* entre ellas es entender el producto.

El centro de gravedad es `accounts.User`. Casi todo cuelga del usuario, porque casi todo lo hace un usuario: publica una plaza, reporta un evento, canjea puntos, cobra dinero. Alrededor de ese núcleo hay **cuatro familias**:

1) Contribuciones geolocalizadas. Un usuario aporta información al mapa — publica una plaza libre (`spaces`) o reporta una alerta vial (`events`). Si esa información resulta cierta (la valida la comunidad), el usuario **gana puntos**. Es el motor que llena el mapa: se premia contribuir *algo que es verdad*, no el mero hecho de publicar.

2) Marketplace. Es donde esos puntos **se gastan**. Tiendas (`Store`) publican productos (`Product`); el usuario canjea puntos y recibe un `RedemptionCode` que enseña en la tienda para llevarse el producto.

3) Sistema de valor. El corazón contable, deliberadamente partido en dos (lo vemos en detalle en la sección 3): `points` lleva los *puntos promocionales* y `credits` lleva el *dinero real* que pasa por Stripe.

4) Capa geoespacial. `maps` traduce coordenadas a direcciones y calcula distancias; `maps` es la difusión en tiempo real por región; `navigation` calcula rutas.

☐ Núcleo / identidad ☐ Dinero real (Stripe) ☐ Puntos promocionales ☐ Geo / tiempo real

```

flowchart TB
    USER(["USER([" 👤 accounts.User  
identidad · GDPR · Pro · phone/documento")])

    subgraph CONTRIB["Contribuciones → generan puntos"]
        SP["SP["spaces  
PaidSpace / FreeSpace  
(polimórfico)"]
        EV["EV["events  
alertas viales"]
        AL["AL["alerts  
radares · ZBE"]
    end
  
```

```

end

subgraph MKT["Marketplace → canjea puntos"]
  ST2["Store · StoreUser"]
  PR["Product · Category"]
  OR["Order · RedemptionCode"]
  TK["Tickets · Disputas · Reviews"]
end

subgraph VALUE["Sistema de valor"]
  PO["points
PointsLot → Hold → Movement
ledger de puntos"]
  CR["credits
EarningAccount · Transaction
euros vía Stripe Connect"]
  WA["wallet
VirtualCard"]
end

subgraph GEO["Geoespacial y tiempo real"]
  MP["maps
geocoding · distancias"]
  MA["mapas
WebSocket por geohash"]
  NV["navigation
rutas"]
end

RES["information_requests
InformationRequest · Dispute"]

USER --> CONTRIB
USER --> MKT
USER --> VALUE
CONTRIB -- "PointsRule" --> PO
OR -- "retiene / captura" --> PO
RES -- "sobre PaidSpace" --> SP
RES -- "hold + capture" --> CR
USER --> RES
CONTRIB --> GEO
MA -. "difunde puntos por región" .-> PO

class USER core;
class SP,EV,AL contrib;
class ST2 PR OR TK mkt·

```

```
class PO,WA pts;
class CR,RES money;
class MP,MA,NV geo;

classDef core fill:#e7ecfd,stroke:#3b5bdb,color:#16202e,stroke-width:1.5px;
classDef contrib fill:#eef1f5,stroke:#c4ccd8,color:#16202e;
classDef mkt fill:#eef1f5,stroke:#c4ccd8,color:#16202e;
classDef pts fill:#ece3ff,stroke:#8a5cf6,color:#16202e;
classDef money fill:#dff3ea,stroke:#0f8a5f,color:#16202e;
classDef geo fill:#eef1f5,stroke:#c4ccd8,color:#16202e;
```

Fig. 2 – Contribuir genera puntos; el marketplace los canjea; pedir la información de una plaza de pago mueve dinero real.

Referencia de apps

APP	TIPO	RESPONSABILIDAD	MODELOS CLAVE
accounts	núcleo	Usuario custom (email + Firebase), coche, GDPR/consentimientos, preferencias, notificaciones, suscripción Pro y datos de perfil (teléfono, documento).	User, Car, UserPreferences, Notification, ConsentRecord, ProZone
credits	dinero	Euros reales en céntimos: cuenta de ganancias del informador vía Stripe Connect, KYC/DAC7, cobros y retiros.	EarningAccount, Transaction, Withdraw, PayoutMethod, PaymentMethod
points	puntos	Puntos promocionales (no convertibles): ledger de lotes, retenciones, libro, reglas, campañas y antifraude.	PointsLot, PointsHold, PointsMovement, PointsRule, Campaign, AntifraudRule
wallet	puntos	Tarjeta/identificador virtual del usuario en el ecosistema.	VirtualCard

marketplace	núcleo	Tiendas, catálogo, pedidos, canje de puntos por productos, reseñas, tickets y disputas de pago.	Store, StoreUser, Product, RedemptionCode, Order, Ticket, PaymentDispute
spaces	geo	Plazas de aparcamiento polimórficas (pagas/gratuitas) que contribuyen puntos; validación comunitaria.	BaseSpace, PaidSpace, FreeSpace, SpaceValidation, SpaceFeedback
information_requests	dinero	Transacción de información sobre una plaza de pago: el solicitante paga (escrow Stripe) y confirma; disputas entre usuarios. <i>(antes llamada reservations)</i> .	InformationRequest, Dispute
events	geo	Alertas viales colaborativas (accidente, policía, corte) que contribuyen puntos; feedback comunitario.	Event, EventFeedback
alerts	geo	Alertas estáticas de infraestructura (radares, cámaras) y zonas de bajas emisiones.	Alert, LowEmissionZone
maps	geo	Servicio de geocoding/distancias con Google Maps, usado por casi todas las apps.	Location + services.google_maps
mapas	geo	Capa de tiempo real: WebSocket que difunde puntos/espacios por región geohash vía	– (consumers, handlers)

navigation	geo	Cálculo y persistencia de rutas con Google Directions.	NavigationRoute
payments	dinero	Capa de vistas/checkout Stripe que lee del ledger de credits (historial de movimientos).	— (views sobre credits)
commons	núcleo	Infraestructura transversal: modelo base UUID, manejo unificado de errores, auth de WebSocket, push, storages. <i>(Ya sin modelos de dominio.)</i>	AbstractUUIDModel

03 Dinero real vs. puntos

Esta es la decisión de diseño más importante de todo el backend, y conviene entenderla bien: **los euros y los puntos viven en apps separadas y nunca se convierten entre sí.** No es una casualidad de organización — es una frontera deliberada por razones legales y contables.

Los **puntos** son promocionales: se ganan gratis contribuyendo y solo sirven para canjear productos en el marketplace. Si fueran convertibles a dinero, serían un instrumento monetario y caerían bajo regulación financiera. Manteniéndolos como "puntos de fidelidad" que nunca salen como efectivo, el producto se mantiene fuera de ese perímetro.

El **dinero**, en cambio, es dinero de verdad, y por eso **Letdem nunca lo custodia**: lo mueve Stripe. El informador cobra en su propia cuenta de Stripe (Stripe Connect), y Letdem solo retiene una comisión. Esto exige verificación de identidad (KYC) y reporte fiscal (DAC7). Confundir ambos mundos —dejar que un punto "valga" un euro en algún rincón del código— sería un problema serio; por eso están tabicados.

€ credits — dinero real	♦ points — promocional
Céntimos gestionados por Stripe Connect .	Saldo que se gana contribuyendo (publicar

El informador cobra en su EarningAccount; Letdem retiene comisión. Requiere KYC y reporte fiscal DAC7. Nunca se genera "gratis".

plaza, reportar evento) y se **canjea** en el marketplace. Ledger FIFO por caducidad; **nunca** se retira como dinero.

El detalle fino del lado de los puntos: no es un simple contador. Es un **ledger** (libro contable de solo-añadir), porque los puntos caducan y hay que saber *qué lote* financia cada gasto. Un PointsLot es un paquete de puntos con fecha de caducidad. Cuando canjeas, se crea un PointsHold que **retiene** puntos (no los gasta aún), y una PointsHoldLine anota de qué lote salen. Solo cuando la tienda *entrega* el producto se capturan de verdad. El PointsMovement es el libro visible para el usuario. Este mismo patrón "retener → capturar" es idéntico al de un pago con tarjeta, y no es casualidad: es la forma correcta de manejar valor que puede revertirse.

Fig. 3 – Dos sistemas de valor paralelos que jamás se cruzan como saldo.

04 Dos flujos que lo explican todo

La mecánica del producto se entiende con dos secuencias. Se parecen —las dos usan un "código" y un paso de confirmación— y precisamente por eso es fácil confundirlas. La diferencia está en **qué tipo de valor mueven**: una mueve dinero, la otra puntos.

A · Transacción de información de una plaza de pago — escrow con Stripe Connect

Aquí no se "reserva" una plaza física. Alguien **paga por la información** de dónde hay una plaza. El **solicitante** paga, pero el dinero no se cobra todavía: se *retiene* (un *hold* de Stripe, igual que la fianza que bloquea un hotel). El solicitante recibe un **código de operación**. Va al sitio, comprueba que la información del **informador** era cierta, y entonces confirma. Solo en ese momento se **captura** el pago y el dinero viaja a la cuenta del informador, menos la comisión de Letdem. Si la información era falsa o hay conflicto, se abre una Dispute. El *hold* protege a ambas partes: el solicitante no paga por información falsa, el informador tiene garantía de cobro si es cierta.

```
sequenceDiagram
    autonumber
    participant Sol as Solicitante
    participant API as information_requests API
    participant St as Stripe
    participant Inf as Informador
```

```

Sol->>API: Pedir info de PaidSpace
API->>St: PaymentIntent (capture_method>manual)
Note over St: fondos retenidos (hold),
              aún no cobrados
St-->>API: intent + código de operación
API-->>Sol: código de operación
Sol->>Inf: usa la info (encuentra la plaza)
Sol->>API: confirmar (info correcta)
API->>St: capture()
St-->>Inf: transfer a EarningAccount
              (menos comisión Letdem)
Note over API: si falla / disputa →
              Dispute

```

Fig. 4A – El dinero se retiene primero y solo se captura al confirmar. Protege a ambas partes.

B · Canje de puntos en el marketplace

El otro flujo es el que la gente suele imaginar cuando oye "muestra un código en la tienda". El usuario canjea **puntos** por un producto y recibe un RedemptionCode. Lo enseña en la tienda; el **dueño de la tienda** lo valida y lo marca como entregado (DELIVERED), y ahí se capturan los puntos retenidos. Si el código caduca sin entregarse, una tarea de Celery libera la retención y devuelve los puntos. Ojo: este flujo (4B) y el de pago (4A) son **distintos** aunque los dos usen un "código" — mueven valor diferente (puntos vs euros) y los confirma quien no debe confundirse (la tienda vs el solicitante).

```

sequenceDiagram
    autonumber
    participant U as Usuario
    participant MK as marketplace API
    participant PT as points (ledger)
    participant ST as Tienda (StoreUser)
    U->>MK: Crear Order (producto por puntos)
    MK->>PT: PointsHold (retiene puntos del lote)
    PT-->>MK: hold ok
    MK-->>U: RedemptionCode (código de canje)
    U->>ST: muestra el código en tienda
    ST->>MK: validar (solo admin de la tienda)
    ST->>MK: entregar (DELIVERED)
    MK->>PT: capturar el hold (puntos consumidos)
    Note over PT: si caduca sin entregar →
    Celery libera el hold (reembolso)

```


Fig. 4B – Los puntos se retienen al canjear y solo se consumen al entregar; si no, Celery los devuelve.

05 Capas transversales

Hay preocupaciones que no pertenecen a una sola app sino que atraviesan todas — autenticación, errores, tareas, tiempo real. Viven sobre todo en commons y en la configuración, y son las que hacen que las 14 apps se sientan como un solo sistema coherente.

Autenticación

Tres mecanismos coexisten: **SimpleJWT** (el principal en REST — el login devuelve un access token firmado, rotación + blacklist), **Firebase** (login social y push) y **DRF Token** (fallback en WebSockets). El middleware de WS los prueba en ese orden hasta identificar al usuario.

Async (Celery)

Redis es la cola. El planificador beat dispara las periódicas: caducar puntos, purga GDPR de cuentas borradas, cerrar contribuciones sin validar, expirar canjes y transacciones de información, reconciliar saldos, y el reporte fiscal DAC7.

Tiempo real

Channels + daphne. MapsConsumer agrupa a los usuarios por región **geohash** y les empuja los puntos cercanos usando Redis directo; RefreshUserConsumer refresca al usuario cuando algo cambia por REST.

API versionada

Todo cuelga de /api/v1/. Un hub central define app_name="v1" y anida un namespace por app, de modo que cada ruta se resuelve como v1:<app>:<name>. Versionar así permite sacar un /v2/ el día de mañana sin romper la app publicada.

Errores unificados

commons.exceptions intercepta cualquier error y lo normaliza a un único contrato {error_code, message, details}, vía middleware + un exception handler de DRF. Así el frontend siempre recibe la misma forma de error, venga de donde venga.

Stripe

credits/providers/stripe/:
crear/capturar/cancelar intents + **dos webhooks** (plataforma y cuentas conectadas), con ~16 eventos mapeados por un registry, incluida la suscripción "Letdem Pro".

Cómo se estratifica una app típica

Cada app sigue la misma anatomía. Leerlas de fuera hacia dentro ayuda a orientarse en cualquiera de las 14:

- **models.py** — el dominio. Casi todo hereda de `AbstractUUIDModel` (aporta `uuid`, `created`, `modified`, `metadata`), así que todos los objetos comparten una identidad y metadatos consistentes.

- **signals.py** — efectos secundarios desacoplados. Cuando se crea una plaza, un signal genera sus puntos, refresca el mapa y programa su caducidad — sin ensuciar la vista.

- **services / módulos de negocio** — la lógica que no es ni de la vista ni del modelo. Muy marcado en `points` (`services`, `antifraud`, `campaigns`) y en `maps`.

- **v1/serializers.py + v1/views.py** — la capa HTTP: validar la entrada y orquestar la respuesta (DRF). `credits` e `information_requests` las expanden a paquetes por sub-dominio.

- **tasks.py** — el trabajo asíncrono (Celery). **ws/** — los consumers de tiempo real.

- **accounts** es la más estratificada de todas: se parte en `modules/{auth, users, admin}`, cada uno con su propio `v1/`.

06 Propuesta de despliegue en un VPS

Hoy el backend corre en un servidor gestionado a mano (procesos arrancados uno a uno, `DEBUG=True`, sin TLS propio ni copias automáticas). Esta sección propone una forma **reproducible, segura y automatizable** de desplegarlo en un VPS —un DigitalOcean Droplet, un Hetzner Cloud, un EC2, cualquiera— pensada para que *un comando* levante todo el sistema y *otro* lo actualice.

La recomendación es **Docker Compose**: cada pieza (web, websockets, worker, base de datos, redis, proxy) es un contenedor declarado en un fichero, con sus versiones fijadas. Eso elimina el "en mi máquina funciona", hace el arranque idempotente y el rollback trivial. Para el tamaño actual, un **único VPS** con todos los contenedores es más que suficiente; el mismo diseño escala después separando la base de datos a un servicio gestionado sin tocar la app.

Topología propuesta (un VPS, Docker Compose)

flowchart TB

USERS["🌐 Internet"] -->|"HTTPS :443"| CADDY

```

subgraph VPS["VPS · Ubuntu 22.04 · Docker Compose"]
    CADDY["Caddy / nginx
    TLS automático (Let's Encrypt)
    /api→web · /ws→ws · /static,/media"]
subgraph APPC["Contenedores de aplicación (misma imagen)"]
    WEB["web · gunicorn
    REST :8000"]
    WS["ws · daphne
    WebSockets :8001"]
    WORKER["worker · celery"]
    BEAT["beat · celery scheduler"]
    end
    CADDY --> WEB
    CADDY --> WS
subgraph INFRA["Infraestructura (contenedores con volúmenes)"]
    PG[("postgres + postgis
    volumen persistente")]
    RD[("redis
    broker · channels · cache")]
    VOL[["volumen media/
    + estáticos"]]]
    end
    WEB --> PG
    WS --> PG
    WORKER --> PG
    BEAT --> PG
    WEB --> RD
    WS --> RD
    WORKER --> RD
    CADDY --> VOL
    WEB -. escribe .-> VOL
    end
    WEB -->|salida| EXT["Stripe · Firebase
    Google Maps · SES/SMTP"]
BK["cron: pg_dump → almacenamiento externo (S3/Spaces)"] -.-> PG

classDef proxy fill:#e7ecfd,stroke:#3b5bdb,color:#16202e;
classDef app fill:#dff3ea,stroke:#0f8a5f,color:#16202e;
classDef inf fill:#ece3ff,stroke:#8a5cf6,color:#16202e;
    class CADDY proxy;
    class WEB,WS,WORKER,BEAT app;
    class PG,RD,VOL inf;

```

Fig. 7 – Un solo host, seis servicios en contenedores. La app se escala replicando web/worker; la RD se externaliza cuando haga falta

Componentes y por qué

Caddy o nginx	Reverse proxy y terminación TLS. Caddy saca y renueva el certificado Let's Encrypt solo (cero config); nginx es la opción conocida si ya se domina. Enruta /api→web, /ws→ws y sirve /static y /media desde volumen.
web (gunicorn)	La REST API WSGI. Se escala subiendo <i>workers</i> dentro del contenedor y, si hace falta, replicando el contenedor detrás del proxy.
ws (daphne)	Los WebSockets ASGI. Contenedor aparte porque su ciclo de vida (conexiones largas) es distinto al de la API.
worker + beat	Celery. worker ejecuta las tareas; beat es el reloj de las periódicas. Van separados para poder reiniciar/escala cada uno sin afectar al otro.
postgres + postgis	La base de datos, con la extensión geoespacial. Datos en un volumen persistente (sobrevive a recrear el contenedor). Es la única pieza con estado crítico → es lo primero a externalizar cuando crezca.
redis	Cola de Celery, channel layer de los WebSockets y caché de sesiones. Ligero; puede compartir host sin problema.
Secretos	Un .env fuera de la imagen (nunca en git): SECRET_KEY, credenciales de BD, claves de Stripe/Firebase/Maps, y — clave — DJANGO_SETTINGS_MODULE=letdem.settings.production con DEBUG=False y ALLOWED_HOSTS.

Flujo de despliegue y actualización

- 1
- #### Preparar el host

Ubuntu al día, instalar Docker + Compose, crear un usuario no-root, y cerrar el firewall (ufw) dejando solo 22, 80 y 443. La BD y Redis **no** se exponen a internet: solo se hablan entre contenedores.
- 2
- #### Construir la imagen

Un Dockerfile con Python 3.11 + GDAL/PostGIS libs + dependencias. La misma imagen sirve para web, ws, worker y beat — solo cambia el comando de arranque. Se etiqueta por versión/commit.

3

Arrancar

`docker compose up -d` levanta los seis servicios. El contenedor web corre `migrate` y `collectstatic` en el arranque. Caddy negocia el TLS solo en el primer golpe.

4

Actualizar (deploy)

CI construye y publica la imagen nueva; en el host, `docker compose pull && docker compose up -d` reemplaza contenedores sin downtime perceptible. **Rollback** = volver a la etiqueta anterior. Las migraciones se aplican en un paso controlado antes de conmutar.

5

Copias y observabilidad

Un cron diario hace `pg_dump` y lo sube a almacenamiento externo (S3/Spaces) con retención. **Sentry** —ya integrado en el código— recoge errores. Logs vía `docker logs/journald`.

Endurecimiento incluido de serie. Frente al estado actual, este diseño trae: **HTTPS** real con renovación automática, **DEBUG=False** con settings de producción (sin fugas de traceback), **backups** automáticos y verificables, BD y Redis **no expuestos** a internet, y despliegues **reproducibles** con rollback en un comando. Migrar es gradual: se puede levantar el stack Docker en paralelo, validar, y conmutar el DNS cuando esté verde.

Dimensionamiento y tráfico orientativo por escalón

Cifras **estimadas** para calibrar cada salto — no son límites exactos, dependen de cuánto pese cada petición. La regla práctica: mientras la CPU media del VPS se mantenga por debajo del ~60 % y la base de datos no sea el cuello de botella, un solo host aguanta. El primer recurso que se satura casi siempre es **PostgreSQL**, por eso es lo primero que se externaliza.

ESCALÓN	USUARIOS / TRÁFICO SUGERIDO	INFRAESTRUCTURA
Arranque	Hasta ~ 5.000 usuarios activos/mes · picos ~ 30–80 req/s · ~1.000 conexiones WebSocket simultáneas.	1 VPS 4 vCPU / 8 GB / 80 GB SSD . Todos los contenedores en el mismo host. Holgado para el tráfico actual.
Primer escalón	~ 5.000–30.000 usuarios activos/mes · ~ 80–250 req/s sostenidos · varios miles de WebSockets.	VPS 8 vCPU / 16 GB , más <i>workers</i> de <code>gunicorn/celery</code> , y PostgreSQL gestionado aparte (backups y réplica de lectura incluidos). La app no cambia.

Escala real	30.000–200.000+ usuarios activos/mes · >250 req/s sostenidos y picos altos.	Varios VPS con la app replicada tras un balanceador ; Redis y PostgreSQL gestionados; media/ en S3 + CDN. El monolito modular aguanta bien este camino antes de plantearse trocearlo.
-------------	---	---

Los WebSockets pesan distinto que las peticiones REST: son conexiones largas. Si el mapa en tiempo real gana tracción, el contenedor ws es el candidato a replicar primero, independientemente de la REST.

07 Credenciales / API keys para producción

Ahora mismo el backend funciona con **mis credenciales de desarrollo** (cuentas de prueba: Stripe en modo test, mi proyecto Firebase, mis keys de Google, etc.). Para que el cliente tenga su **propio entorno de producción**, debe dar de alta **sus propias cuentas** en cada servicio externo y generar sus claves. Las de dev **no se reutilizan en producción** — son de prueba y, por seguridad, no deben salir de mi entorno.

Todos estos valores viven en el fichero `.env` del backend (que **nunca** va a git). Aquí se listan solo los **nombres de variable y qué pedir a cada proveedor**, no los valores.

A · Servicios externos — el cliente crea la cuenta y genera la clave

SERVICIO	PARA QUÉ	VARIABLES (.ENV BACKEND)	QUÉ DÓN
Stripe	Pagos y cobros a informadores (Connect)	STRIPE_SECRET_KEY · HTTP_STRIPE_SIGNATURE_ACCOUNTS · HTTP_STRIPE_SIGNATURE_CONNECTED_ACCOUNTS	Cue del mo sk_ (sec whs cad 2 w (pla cue con pk_ (pul en l app no a

Firebase	Login social + push notifications	FIREBASE_CREDENTIALS · FIREBASE_SERVICE_CONFIG	Proy Fire clien de s acco (Con → C serv Ade app ser (An Goo Inf (iOS SHA
Google Maps	Geocoding, rutas, distancias	GOOGLE_MAPS_API_KEY	API Goc con Dir Dist Mat hab fact acti rest app móv
Mapbox	Capa de mapa	MAPBOX_ACCESS_TOKEN	Acc la c Map clien
OneSignal	Envío de push	ONESIGNAL_APP_ID · ONESIGNAL_API_KEY · ONESIGNAL_APP_KEY · ONESIGNAL_USER_KEY	App One clien + R key: tam app

AWS S3	Almacenamiento de media (imágenes)	<code>AWS_ACCESS_KEY_ID</code> · <code>AWS_SECRET_ACCESS_KEY</code> · <code>AWS_STORAGE_BUCKET_NAME</code> · <code>AWS_S3_BUCKET</code> · <code>AWS_S3_REGION</code>	Usu (cor solc buc buc clie regi
AWS SES	Email transaccional en producción	<code>AWS_SES_REGION_ENDPOINT</code> · <code>AWS_SES_REGION_NAME</code>	Dor veri SES sanc usa misi crec AW: Alte SM sig
SMTP (email)	Email por SMTP (dev o si no usan SES)	<code>EMAIL_HOST</code> · <code>EMAIL_PORT</code> · <code>EMAIL_HOST_USER</code> · <code>EMAIL_HOST_PASSWORD</code> · <code>FROM_EMAIL</code>	Cre pro SM (Ser Gm rem
Sentry	Monitorización de errores (opcional)	<code>SENTRY_DSN</code>	DSN proy del deja simi rep

B · Configuración propia del despliegue — no se pide a un tercero, se define

<code>SECRET_KEY</code>	Generar uno nuevo, único y secreto para producción (no el de dev).
-------------------------	---